adapter.py

`src/moe_pruning/models/olmoe/adapter.py · 324 lines · module moe_pruning.models.olmoe.adapter`

What it does

Everything specific to OLMoE in transformers 5: decoder-only, every layer sparse, softmax top-8 routing over 64 experts whose weights are stored as two fused 3D tensors per layer.

How it works

- Discovers `model.layers.{i}.mlp` blocks; layer ids look like `layers.3`; no capacity limit.
- Because experts are fused, `replace_experts_in_layer()` swaps the whole container for a `PrunedFusedExperts` that keeps only the retained experts' rows and serves removed experts through their replacement module.
- Instrumentation hooks the post-attention norm (residual), the MoE block and the router; un-gated expert outputs are recomputed from the same weights because the fused kernel does not expose them.
- The reroute mask re-runs top-k over the remaining experts, using the original probabilities as gates.
- `load_original_expert()` reads an expert from the Hub layout (`experts.{e}.gate_proj` etc.) or the fused layout.
- `removed_weight_violations()` checks that stored fused tensors have exactly the retained number of rows.

Defined in this file

Extracted from the source code (classes, public functions and methods).

Name	Description
`class OlmoeAdapter`	MoEAdapter for OLMoE with transformers 5 fused experts.
`.discover_layers(model)`	Find all sparse MoE layers by inspecting the loaded model.
`.get_moe_module(model, layer)`	The sparse MoE module of a layer.
`.get_router(model, layer)`	The router module of a layer.
`.get_expert(model, layer, expert_id)`	Module computing one expert (original, view or replacement).
`.build_original_expert(model, layer)`	New (uninitialised) expert module of the model's original expert class.
`.expert_state_prefix(layer, expert_id)`	State-dict key prefix of an expert slot (ends with a dot).
`.replace_experts_in_layer(model, layer, modules)`	Install `modules` in their expert slots; return the number of removed parameters.
`.set_expert(model, layer, expert_id, module)`	Install `module` in the expert slot and return the previous module.
`.removed_weight_violations(state_shapes, layer, removed)`	Names of stored tensors that still hold weights of removed experts.
`.validate_loaded(model, layers)`	Called by the checkpoint loader after weights are loaded; raise on inconsistency.
`.load_original_expert(reader, model, layer, expert_id)`	Read one expert from the hub layout (per expert) or the fused layout.
`.route(model, layer, moe_input, apply_mask)`	Return (expert_index [B,S,K], gate [B,S,K]) exactly as used by the layer.
`.set_routing_mask(model, layer, expert_ids)`	Make `expert_ids` unreachable: their tokens go to the best remaining expert.
`.get_routing_mask(model, layer)`	Experts currently made unreachable by the reroute mask.

Name	Description
`.run_moe_layer(model, layer, moe_input)`	Execute only the sparse MoE module on `moe_input` ([B,S,D]).
`.instrument(model, layers, callback)`	Call `callback` with a `MoEForwardRecord` per layer forward.
`.describe(model)`	Machine-readable architecture report.

Connections

Uses: `moe_pruning.model.architecture`, `moe_pruning.models.olmoe.experts`

Used by: `src/moe_pruning/models/olmoe/__init__.py`